

Machine Learning

Semester 2026-III

Final Project — Multi-Agent Learning Systems

Eng. Carlos Andrés Sierra, M.Sc.

Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

Congratulations on completing six intensive workshops developing machine learning techniques across different paradigms. This final project represents the capstone of your machine learning journey: building a ****multi-agent learning system**** that applies all the concepts and techniques you have mastered to solve a complex, domain-specific challenge.

Multi-agent systems represent a frontier of modern AI research. Unlike single-agent systems, multi-agent systems model environments where multiple intelligent entities interact, compete, cooperate, and learn simultaneously. Your task is to design and implement a multi-agent system appropriate to your assigned domain, integrating deep learning, reinforcement learning, and classical machine learning approaches. The system should demonstrate emergent behavior, strategic interaction, and measurable learning across multiple agents.

Final Project Definition:

A multi-agent learning system is a computational environment populated by two or more autonomous agents that perceive their environment, take actions, receive rewards, and learn to improve their performance. Agents may be competitive (adversarial), cooperative (aligned objectives), or mixed. Deep learning provides representation learning and perception; reinforcement learning drives agent learning and strategy optimization; classical machine learning establishes baselines and heuristic strategies. Your final project integrates all techniques from Workshops 1-6 to create a cohesive multi-agent system that solves a meaningful problem in your domain.

Carlos Andrés Sierra, Computer Engineer, M.Sc. in Computer Engineering, Full-time Adjunct Professor at Universidad Distrital Francisco José de Caldas.

Any comment or concern about this document can be sent to Carlos A. Sierra at: *cavir-guezs@udistrital.edu.co*.

Project Overview and Core Principles

- **Domain-Specific Problem:** Each team develops a multi-agent system within their assigned domain. The system should address a realistic problem that naturally involves multiple agents with different objectives or roles.
- **Multi-Agent Architecture:** Design at least 2–3 distinct agent types with different roles, objectives, and strategies. Agents should learn over time and adapt to each other’s behavior.
- **Integration of All Workshops:** The system should demonstrably use techniques from all six workshops: baselines (W1), deep learning architectures (W2), RL algorithms (W3), autoencoders (W4), generative models (W5), and domain knowledge (W6).
- **Emergent Behavior:** The system should exhibit emergent properties that arise from agent interactions—behaviors or outcomes not explicitly programmed but learned through multi-agent dynamics.
- **Measurable Learning:** Track agent performance, learning curves, strategy evolution, and system-level metrics over time. Agents should improve as they interact.
- **Reproducibility and Clarity:** Full code, training logs, and clear documentation enabling reproduction of all results.

Domain-Specific Capstone Projects

1. Team 1 — Real Estate Market Simulator

Problem: Design a multi-agent real estate market where buyers, sellers, and brokers interact, negotiate, and learn optimal pricing and bidding strategies.

- **Agents:** Buyer agents (learn what to bid based on property features and market conditions), Seller agents (learn reservation prices), Broker agents (learn commission strategies).
- **RL Component:** Each agent uses RL to optimize profit/utility over many transactions. W3 directly applies: Q-learning or policy gradients for strategy learning.
- **Deep Learning Component:** CNN or MLP to predict property values from features. W2 neural networks learn price-feature relationships. Autoencoders (W4) learn property feature representations.
- **GAN Component:** W5 GANs generate synthetic properties to test agent strategies on diverse scenarios.
- **Baseline:** W1 heuristic pricing rules (comparable market analysis, simple rules).
- **Emergent Behavior:** Market prices fluctuate based on agent behavior; rational bubbles or crashes may emerge from collective learning.
- **Medical Domain Application (W6):** Adapt to medical real estate market (hospitals, clinics, research facilities) where medical infrastructure and compliance add domain constraints.

2. Team 2 — Customer Retention Learning Game

Problem: Design a multi-agent game where customer agents decide to stay or leave, and company retention agents learn optimal retention strategies.

- **Agents:** Customer agents (decide retention vs. churn based on offers and satisfaction), Company agents (decide retention strategies, incentives, pricing).
- **RL Component:** Both sides use RL to learn optimal strategies. W3: mini-max game-theoretic learning.
- **Deep Learning:** W2 RNN/LSTM predicts customer behavior trajectories. W4 autoencoders learn customer segment representations.
- **GAN Component:** W5 synthetic customer scenarios for rare churn patterns or new market segments.
- **Baseline:** W1 fixed retention rules (rule-based heuristics).
- **Emergent Behavior:** Arms race dynamics; company strategies evolve to counter customer expectations; customer responses adapt.

- **Medical Application (W6):** Patient retention in healthcare (preventing patient churn to competitors; learning optimal patient engagement strategies).

3. Team 3 — Adversarial Image Generation and Classification

Problem: Design a multi-agent system where generator agents learn to create images and classifier agents learn to distinguish real from fake, in an adversarial loop.

- **Agents:** Generator agent (creates synthetic images), Discriminator agents (multiple classifiers with different architectures), Evaluator agent (assesses quality).
- **RL Component:** W3 RL formulation: generator learns to fool discriminators; discriminators learn to detect fakes. Reward shaping crucial.
- **Deep Learning:** W2 W5 core: CNNs for classification, GANs for generation. Multiple discriminators create a richer learning environment.
- **Baseline:** W1 simple image features + classical classifiers.
- **Emergent Behavior:** Discriminators develop diverse detection strategies; generator learns to create images that fool specific classifiers; arms race dynamics.
- **Medical Application (W6):** Generate and classify medical images; evaluate whether synthetic medical images are indistinguishable from real.

4. Team 4 — Multi-Agent Dialogue System

Problem: Design a dialogue system with multiple conversational agents (user simulator, response generator, evaluator) that learn to communicate effectively.

- **Agents:** User agent (simulates realistic user utterances and preferences), Dialogue agent (learns to generate contextually appropriate responses), Evaluator agent (judges dialogue quality).
- **RL Component:** W3 dialogue agents use RL to optimize for successful conversation outcomes (user satisfaction, task completion). Reward: dialogue success metrics.
- **Deep Learning:** W2 Transformers/RNNs for dialogue generation and understanding. W4 autoencoders learn dialogue state representations (context, intent).
- **GAN Component:** W5 adversarial text generation: agents learn to generate realistic or adversarial utterances.
- **Baseline:** W1 template-based responses, retrieval-based baseline.
- **Emergent Behavior:** Dialogue patterns evolve; user agent learns to exploit system weaknesses; dialogue agent develops adaptive strategies.
- **Medical Application (W6):** Medical chatbot for patient intake, symptom triage, or patient education; evaluate clinical accuracy and safety.

5. Team 5 — Multi-Agent Game Playing

Problem: Design a multi-agent game (cooperative or competitive) where agents learn to play optimally through self-play and mutual learning.

- **Agents:** Multiple game-playing agents, each using RL to learn policy and value functions. Agents compete or cooperate toward shared goals.
- **RL Component:** W3 core: DQN, policy gradient, or actor-critic for each agent. Multi-agent RL with self-play (agents train against increasingly skilled versions of themselves).
- **Deep Learning:** W2 neural networks learn state evaluation and value functions. W4 autoencoders learn game state representations.
- **GAN Component:** W5 optional: adversarial agents or generative models for curriculum learning (generate diverse game scenarios).
- **Baseline:** W1 rule-based heuristic agents (minimax, greedy strategies).
- **Emergent Behavior:** Agents discover novel strategies, form emergent team behaviors, develop counter-strategies in arms race.
- **Medical Application (W6):** Model medical team coordination (surgeon, anesthesiologist, nurses learning optimal collaborative strategies) or treatment decision-making under uncertainty.

6. Team 6 — Distributed Anomaly Detection

Problem: Design a multi-agent monitoring system where distributed detector agents communicate and coordinate to identify anomalies in a large system.

- **Agents:** Detector agents (monitor different subsystems or data streams), Coordinator agent (aggregates alerts, decides response), Learner agent (updates anomaly model).
- **RL Component:** W3 RL for detector coordination: learn when to alert (avoid false alarms), how to allocate monitoring resources, optimal response strategies.
- **Deep Learning:** W2 neural networks for anomaly scoring. W4 autoencoders learn normal patterns; reconstruction error signals anomalies.
- **GAN Component:** W5 GANs generate synthetic anomalies for edge case testing and detector training.
- **Baseline:** W1 statistical baselines (threshold-based detection, clustering).
- **Emergent Behavior:** Detectors specialize; coordinator learns alert prioritization; system adapts to changing anomaly patterns.
- **Medical Application (W6):** Patient monitoring network with distributed sensors; agents learn optimal alert thresholds; early warning for deterioration.

7. Team 7 — Recommendation Ecosystem

Problem: Design a recommendation system with multiple competing recommendation agents that learn from user interactions and coordinate to provide high-quality recommendations.

- **Agents:** Content-based recommender, Collaborative recommender, Context-aware recommender, Quality evaluator (judge recommendations). Agents compete for user attention or cooperate toward user satisfaction.
- **RL Component:** W3 RL for each recommender to optimize user satisfaction while managing explore-exploit tradeoff and avoiding filter bubbles.
- **Deep Learning:** W2 deep collaborative filtering learns user-item embeddings. W4 autoencoders learn latent user preference factors.
- **GAN Component:** W5 GANs generate synthetic users or items for testing recommender robustness.
- **Baseline:** W1 matrix factorization, simple collaborative filtering.
- **Emergent Behavior:** Recommenders develop complementary strategies; system learns to balance diverse recommendations; user preferences shape agent strategies.
- **Medical Application (W6):** Treatment recommendation system; agents recommend based on clinical evidence, patient preferences, outcomes; learn from patient compliance and results.

8. Team 8 — Iterative Text Generation with Editor Agents

Problem: Design a multi-agent text generation system where writer agents generate text and editor agents iteratively refine, learn quality criteria, and improve output.

- **Agents:** Writer agent (generates text), Editor agents (multiple editors with different quality criteria: fluency, coherence, factuality), Evaluator agent (judges final quality).
- **RL Component:** W3 RL for writer to generate text that satisfies multiple editor criteria. Rewards from editors guide learning.
- **Deep Learning:** W2 Transformers for language generation. W4 autoencoders learn semantic representations for editing and refinement.
- **GAN Component:** W5 adversarial training: writer learns to generate text that passes editor scrutiny; editors learn to detect weaknesses.
- **Baseline:** W1 template-based text, simple language models.
- **Emergent Behavior:** Writer develops strategies to satisfy multiple editors; editors specialize in different quality dimensions; iterative improvement loop.
- **Medical Application (W6):** Medical report generation; writer generates clinical reports, editor agents check accuracy/completeness/clarity, system learns to produce clinical-grade documentation.

Integration of Workshop Concepts

Workshop	Role in Multi-Agent System
W1: Baselines	Heuristic agent policies; simple baseline strategies
W2: Deep Learning	Perception, representation learning, feature extraction
W3: Reinforcement Learning	Agent learning algorithms (DQN, policy gradient, game theory)
W4: Autoencoders	State representation learning; dimensionality reduction
W5: Generative Models	Synthetic data generation; adversarial training
W6: Medical Domain	Domain-specific agent constraints, evaluation, deployment

Each agent in your multi-agent system should integrate these components:

- **Perception:** Use W2 deep learning to process observations (images, text, tabular data).
- **Representation:** Use W4 autoencoders or W2 learned features to build compact state representations.
- **Learning:** Use W3 RL algorithms (Q-learning, policy gradient, etc.) to optimize agent policies.
- **Training:** Use W5 generative models to augment training scenarios with synthetic data.
- **Baseline Comparison:** Use W1 classical ML heuristics as baselines to measure RL improvement.
- **Domain Validation:** Use W6 domain knowledge to validate that learned behaviors make sense.

Required Deliverables

1. System Design Document (5–10 pages):

- **Problem formulation:** Why is this problem naturally multi-agent? What are agent objectives?
- **Agent architecture:** How many agents? What are their roles, observation spaces, action spaces, reward functions?
- **Integration strategy:** How does each workshop concept contribute to your system?
- **Expected emergent behaviors:** What non-obvious behaviors might arise from agent interaction?

2. Implementation and Training (Code):

- `src/agents/` — implementation of all agent types
- `src/environment/` — environment simulation and agent-environment interaction
- `src/train.py` — multi-agent training loop
- `src/evaluate.py` — comprehensive evaluation metrics
- Each agent should use deep learning (neural network) + RL components
- Full reproducibility with random seeds, hyperparameters documented

3. Experimental Results and Analysis (10–15 pages):

- **Learning Curves:** Plot performance of each agent over training episodes. Show convergence, learning dynamics.
- **Strategy Evolution:** How do agent strategies change over time? What patterns emerge?
- **Inter-Agent Dynamics:** How do agents respond to each other? Evidence of mutual adaptation?
- **Emergent Behaviors:** Document any unexpected behaviors arising from agent interactions.
- **Baseline Comparison:** Compare RL agents against W1 heuristic baselines. Quantify improvement.
- **Ablation Study:** Remove components (RL, deep learning, GANs) and measure impact.
- **Failure Analysis:** What scenarios cause agent failure? Why?

4. Multi-Agent Analysis Report (8–10 pages):

- **Emergent Properties:** Describe system-level behaviors that arise from agent interactions.

- **Game Theory:** Analyze agent interactions through game-theoretic lens (Nash equilibrium, Pareto optimality, etc.).
- **Scalability:** How does system performance scale with number of agents?
- **Robustness:** How do agents perform against adversarial agents or distribution shifts?
- **Generalization:** Do learned strategies transfer to new scenarios?

5. **Domain Integration and Validation (5–8 pages):**

- How does your system connect to real-world problems in your domain?
- Does the system produce outputs meaningful to domain experts?
- What are limitations in translating to real deployment?
- For medical teams (W6): How would this integrate with actual clinical systems?

6. **Final Report (Combined PDF):** Integrate all sections above into a single comprehensive document (40–50 pages recommended). Include:

- Abstract
- Introduction (problem motivation, related work)
- Background (domain overview, multi-agent RL foundations)
- System Design (architecture, agents, environment)
- Implementation (technical details, algorithms)
- Experiments (methodology, results, analysis)
- Multi-Agent Dynamics (emergent behaviors, game theory)
- Domain Integration (validation, real-world applicability)
- Conclusions and Future Work
- References (minimum 30 peer-reviewed citations)

7. **Code Repository:**

- Clean, well-documented code
- Comprehensive README with setup and reproduction instructions
- All checkpoints and trained agent policies
- Training logs and results
- Tagged with `v1.0-final`

8. **Live Demonstration or Video:**

- Video (5–10 minutes) showing agents learning and interacting
- Visualizations of strategy evolution, emergent behaviors
- Narration explaining what is happening and why it matters

Evaluation Criteria

Component	Weight
System Design (multi-agent problem formulation)	15%
Technical Implementation (code quality, completeness)	20%
Learning and Convergence (clear evidence of agent learning)	20%
Emergent Behaviors (non-obvious system properties)	15%
Workshop Integration (all 6 workshops demonstrably used)	15%
Analysis and Insights (understanding of multi-agent dynamics)	10%
Domain Relevance and Validation	5%
Total	100%

Key Success Criteria:

- **Multiple Learning Agents:** At least 2–3 distinct agent types learning over time
- **Evidence of Learning:** Clear performance improvement over episodes/iterations
- **Inter-Agent Dynamics:** Agents adapt to each other; strategies coevolve
- **Emergent Behavior:** System exhibits properties not explicitly programmed
- **Workshop Integration:** Demonstrable use of concepts from all 6 workshops
- **Reproducibility:** All results reproducible from submitted code
- **Analysis Depth:** Thoughtful reflection on what agents learn and why

Important Guidelines and Considerations

- **Multi-Agent RL is Complex:** Training multiple agents simultaneously introduces non-stationarity (agents' policies change, making others' learning targets unstable). Expect slower convergence than single-agent RL. Use techniques like experience replay, target networks, and policy smoothing.
- **Emergent Behavior Cannot be Forced:** You cannot simply program emergent behaviors; they arise from agent interactions. Design the environment and reward functions to enable emergence, then observe what happens.
- **Game Theory Matters:** Think about your system through game-theoretic lens. Are agents in a cooperative game, competitive game, or mixed? What equilibria might they reach?
- **Visualization is Critical:** Multi-agent systems are hard to debug. Visualize agent behaviors, learned strategies, interactions. This is essential for understanding what is happening.
- **Separate Training and Evaluation:** Train agents in a training environment, then evaluate on held-out test scenarios. Avoid data leakage between training and evaluation.
- **Document Hyperparameters:** Multi-agent RL is sensitive to hyperparameters. Document all hyperparameters, learning rates, reward scaling, exploration rates, etc. Multiple runs with different seeds are essential.
- **Negative Results are Valuable:** If agents fail to learn, or if emergent behaviors are undesirable, this is scientifically valuable. Document why failure occurred and what you learned.
- **Scale Appropriately:** Start with simple scenarios (few agents, simple environments) and scale up. Don't try to build the most complex system immediately.
- **Baseline Comparison is Essential:** Compare RL agents against W1 heuristic baselines. This quantifies the value of learning.
- **Think About Fairness and Ethics:** In competitive multi-agent systems, do some agents systematically win or lose? Is this inherent to the problem or due to unfair learning dynamics? In domains like recommendations, discuss filter bubbles and information diversity.

Submission Requirements

- **Final Report PDF:** Naming convention: `groupN_final_project.pdf`
- **Code Repository:** Tagged with `v1.0-final`, full reproducibility documentation
- **Supplementary Materials:** Video demonstration, training logs, visualizations
- **Presentation:** 30-minute presentation per team covering system design, key results, and lessons learned
- **All results must be reproducible** from submitted code using only the README instructions

This final project represents the culmination of your machine learning education. You have learned individual techniques across six workshops. Now you must synthesize them into a coherent multi-agent system that exhibits emergent intelligence. Multi-agent systems are frontier research in AI—complex, unpredictable, and beautiful. The behaviors that emerge from your agents learning and interacting are genuine discoveries, not predetermined outcomes. Approach this project with curiosity, careful observation, and rigorous analysis. Good luck, and enjoy the magic of watching multiple learning agents discover strategies together!